Bazar.com – Lab 1 Final Design Report

Course: Distributed and Operating Systems
Project: Multi-tier Online Book Store
Contributors: Batool Masri and Raana Saadaldein
Technologies: Python, Flask, SQLite, REST APIs, Docker, Docker Compose

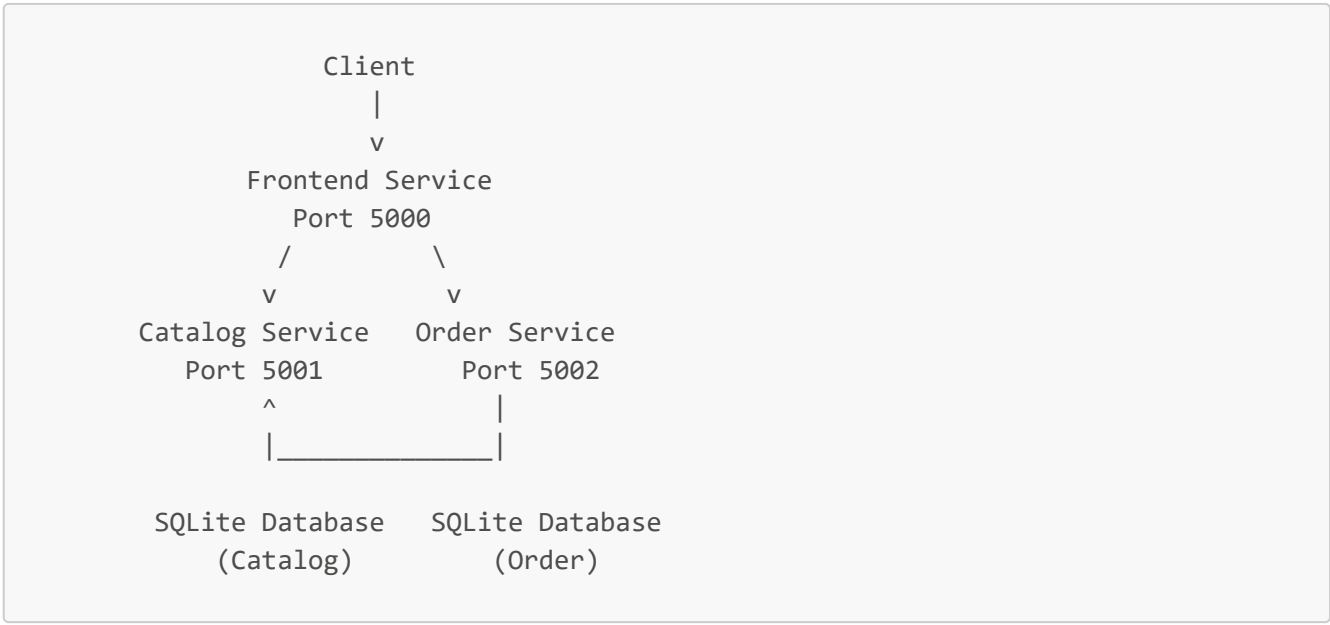
1. System Architecture

Bazar.com is a distributed online bookstore implemented using three independent microservices: Frontend, Catalog, and Order. The services communicate through HTTP REST APIs and exchange data in JSON format.

The Frontend Service receives client requests and forwards them to the appropriate backend service. The Catalog Service manages book information and inventory, while the Order Service processes purchases and stores order records.

The architecture follows a multi-tier design with a client-facing Frontend and separate backend services.

Service communication:



Each service runs in a separate Linux Docker container.

2. Microservices and REST APIs

Frontend Service

The Frontend Service runs on port 5000 and exposes three endpoints:

Method	Endpoint	Function
GET	`/search/<topic>`	Search for books
GET	`/info/<item_id>`	Retrieve book information
POST	`/purchase/<item_id>`	Forward purchase requests

The Frontend uses HTTP requests to communicate with Catalog and Order. Backend URLs are configured using environment variables.

Catalog Service

The Catalog Service runs on port 5001 and stores book information in SQLite.

Each book contains an ID, title, topic, price, and available quantity.

The service supports searching, retrieving book information, updating prices and quantities, decrementing stock, and restoring stock after failed purchases.

Order Service

The Order Service runs on port 5002 and processes purchases.

When a purchase request is received, the service generates a unique transaction ID and requests an inventory decrement from Catalog.

If stock is available, the purchase is saved in the Order SQLite database. Successful purchases return the order ID, book ID, price, remaining stock, and transaction ID.

The service also provides a health endpoint for checking its availability.

3. Data Persistence and Transaction Management

Catalog and Order maintain separate SQLite databases.

Docker named volumes preserve both databases when containers are stopped or recreated.

Inventory updates use SQLite transactions to protect against concurrent modifications.

The Catalog decrement operation uses `BEGIN IMMEDIATE` to coordinate database writes and prevent overselling.

Unique transaction IDs support idempotent stock operations, ensuring that repeated requests with the same transaction ID do not decrement inventory multiple times.

If stock is decremented but saving the order fails, the Order Service attempts to restore the stock through Catalog's compensation endpoint.

This mechanism provides application-level compensation rather than a fully atomic distributed transaction.

4. Docker Deployment

Docker Compose orchestrates the three services.

Service	Port	Persistent Volume
Frontend	5000	None
Catalog	5001	`catalog_data`
Order	5002	`order_data`

Docker provides isolated Linux environments, while Compose configures networking, environment variables, and persistent storage.

The complete application can be started using:

```
docker compose up --build -d
```

Services communicate through Docker DNS names such as `catalog` and `order`.

5. Testing and Results

The system was tested using REST requests, Python scripts, SQLite queries, and Docker commands.

Functional and integration testing:

- Catalog search and book information requests succeeded.
- Frontend-to-Catalog and Frontend-to-Order communication succeeded.
- Purchase requests created persistent order records.
- Docker volumes preserved data after container recreation.
- Invalid book requests returned HTTP 404.

Concurrency testing:

Ten concurrent purchase requests were sent for a book with four available copies.

Results:

Result	Count
Successful purchases (HTTP 200)	4
Out-of-stock responses (HTTP 409)	6
Other errors	0
Final stock	0

All four new successful purchases were verified in the Order SQLite database.

Failure handling:

When Catalog was stopped, Frontend returned HTTP 503. After Catalog restarted, information requests succeeded with HTTP 200.

When Order was stopped, purchase requests returned HTTP 503. After restarting Order, its health endpoint returned HTTP 200.

A simulated Order database failure also verified successful stock compensation.

Detailed test results are available in the `docs/` directory.

6. Design Decisions, Limitations, and Improvements

Flask was selected because it is lightweight and suitable for implementing REST microservices.

SQLite provides simple file-based persistence without requiring a separate database server.

Docker Compose simplifies deployment and communication between independently running services.

Separating services improves modularity but introduces network overhead and partial-failure scenarios.

The current system has several limitations:

- Flask's built-in server is intended for development rather than production.
- Docker Compose startup ordering does not guarantee service readiness.
- Network interruptions may produce uncertain transaction outcomes.
- Compensation does not provide complete distributed transaction guarantees.

Potential improvements include automated retries, stronger transaction recovery, centralized logging, health checks, and production-ready servers.

7. Conclusion

Bazar.com demonstrates a multi-tier distributed bookstore using three REST microservices.

The completed implementation supports book searches, inventory management, purchase processing, persistent storage, and concurrent requests.

Testing confirmed correct behavior for the evaluated concurrency, persistence, and service-failure scenarios.

The project illustrates key distributed systems concepts, including service communication, independent deployment, data consistency, concurrency control, and failure handling.